

INGEGNERIA DEL SOFTWARE · INGEGNERIA
INFORMATICA · UNISA

Ingegneria dei Requisiti

Appunti di ripasso e materiale d'esame — Lezione 3

Elicitazione

Analisi & priorità

Definizione dei requisiti

SRS

Tracciabilità

Prototipazione

Fonte del corso

Slide "Lezione 3 — Ingegneria dei Requisiti", P. Ritrovato — N. Capuano, DIEM,
Università di Salerno.

Testo di riferimento: I. Sommerville, *Ingegneria del Software*, 10^a ed., Pearson, Capitolo
4 (tranne 4.6).

Rielaborato e ampliato per lo studio e la preparazione dell'esame.

Indice

1. Introduzione all'ingegneria dei requisiti

Cos'è un requisito · Le due funzioni dei requisiti · Le attività dell'ingegneria dei requisiti · Perché documentare i requisiti

2. Elicitazione dei requisiti

Fonti e stakeholder · Requisiti di alto livello e di dettaglio · Problematiche di elicitazione · Interviste · Analisi etnografica

3. Analisi dei requisiti

Categorizzazione funzionali/non funzionali · Evitare le ambiguità · Le sei dimensioni dei requisiti · Priorità · Coerenza e completezza

4. Definizione dei requisiti

Linguaggio naturale · Linguaggio naturale strutturato (IPO, template) · Casi d'uso · Tracciabilità · Prototipazione · Checklist di revisione

5. Specifica e accettazione dei requisiti

Documento SRS · Accettazione da parte del cliente

6. Riepilogo per l'esame

Mappa concettuale in breve · Domande di autoverifica con risposta

1 · Introduzione all'ingegneria dei requisiti

1.1 Cos'è l'ingegneria dei requisiti

DEFINIZIONE L'**ingegneria dei requisiti** è l'insieme delle attività necessarie alla definizione e alla comprensione: (a) delle **funzionalità** che un cliente richiede a un sistema; (b) dei **vincoli** sotto i quali il sistema viene sviluppato e opera.

I **requisiti** sono quindi le descrizioni delle funzionalità e dei vincoli del sistema: descrivono **cosa** dovrebbe fare il sistema software, non **come** dovrà essere realizzato. Questa distinzione cosa/come è la stessa vista nella Lezione 1 tra requisiti e progettazione: i requisiti restano "a monte" delle decisioni implementative.

1.2 Che cos'è un requisito?

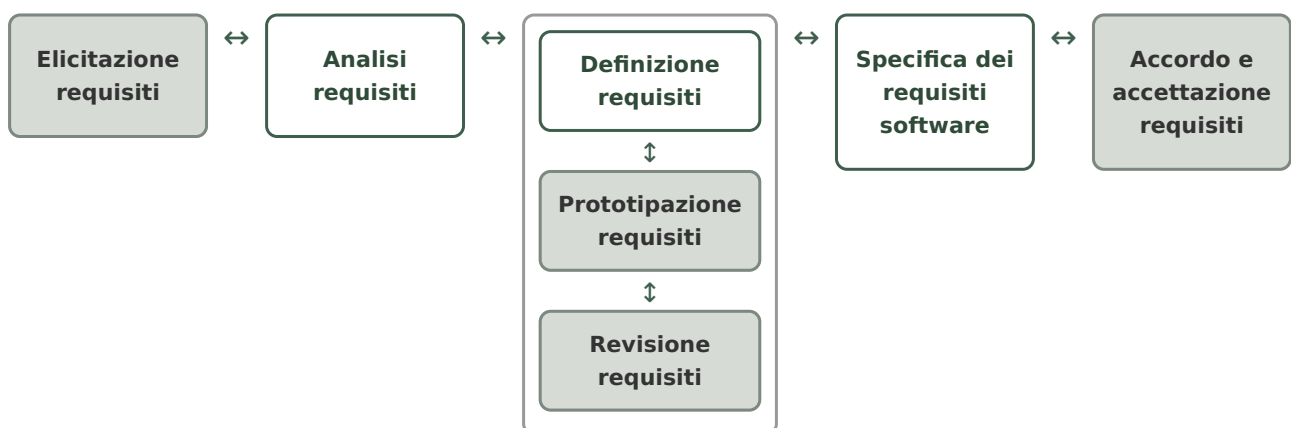
Un requisito può variare moltissimo di livello di dettaglio: va da una **dichiarazione astratta di alto livello** di una funzionalità o di un vincolo, fino a una **specifica formale matematica dettagliata**. Questa variabilità non è un difetto: dipende dallo scopo per cui il requisito viene scritto.

LE DUE FUNZIONI POSSIBILI DI UN REQUISITO

- **Base per un'offerta/contratto da negoziare** — in questo caso deve restare **aperto all'interpretazione**, perché il fornitore deve avere margine per proporre soluzioni diverse
- **Base del contratto stesso** — in questo caso deve essere **definito in dettaglio**, perché diventa il criterio con cui si verifica se il fornitore ha rispettato gli accordi

1.3 Le attività dell'ingegneria dei requisiti

Le attività principali sono collegate tra loro in un ciclo con andirivieni, non in una sequenza rigida:



☒ Coinvolge il cliente/
utente

☐ Attività prevalentemente interna al
team

COME LEGGERE LO SCHEMA Le frecce a doppio senso non sono un dettaglio grafico: indicano che il processo **non è lineare** (coerentemente con quanto visto sul processo software nelle Lezioni 1-2). Ad esempio, mentre si scrive la specifica può emergere che un requisito era stato mal definito, il che riporta indietro all'analisi o persino a una nuova elicitazione. Da notare anche che **tre delle sei attività coinvolgono direttamente il cliente** (elicitazione, prototipazione, revisione, accordo/accettazione): l'ingegneria dei requisiti non è mai un lavoro isolato del solo team tecnico.

1.4 Perché documentare i requisiti

- Servono **requisiti chiari** per le attività di progettazione e implementazione
- Servono a creare **casi e scenari di test**, soprattutto quando il team di test è separato dagli sviluppatori
- Sono necessari per creare **materiale formativo** per gli utenti e documenti di supporto/manutenzione
- Consentono di **suddividere un progetto di grandi dimensioni**, dare priorità ai rilasci e semplificare la gestione del progetto

LA FRASE PIÙ IMPORTANTE DELLA LEZIONE Una delle ragioni principali del **fallimento di un progetto software** sono le specifiche dei requisiti **incomplete, errate o mal comprese**.

L'ESEMPIO CLASSICO DA CONOSCERE A lezione viene mostrata la celebre vignetta (molto citata nei corsi di ingegneria del software e nella project management in generale) che rappresenta un'altalena appesa a un albero, disegnata in modo completamente diverso da ciascun soggetto coinvolto nel progetto: come l'ha spiegata il cliente, come l'ha capita il project leader, come l'ha progettata l'ingegnere, come l'ha scritta il programmatore, come l'ha descritta il reparto vendite, come è stata documentata, come è stata installata, come è stata fatturata al cliente, come l'ha supportata l'helpdesk — e infine cosa serviva *davvero* al cliente (spesso qualcosa di molto più semplice di tutte le versioni precedenti). Il punto didattico è che **ogni passaggio della catena di comunicazione introduce una propria interpretazione** dello stesso requisito, ed è esattamente questo che l'ingegneria dei requisiti cerca di prevenire attraverso documentazione chiara e verificata a ogni fase.

2 · Elicitazione dei requisiti

2.1 Fonti dei requisiti e stakeholder

I requisiti principali vengono di solito forniti dal cliente/utente, ma questo canale ha dei limiti strutturali:

- Gli utenti a volte comprendono solo i requisiti relativi alle **proprie mansioni**, non alla visione d'insieme
- Le **motivazioni e gli obiettivi aziendali** non sono sempre chiari ai singoli utenti, e vanno individuati per stabilire le priorità
- I requisiti dichiarati da utenti e clienti possono essere **contraddittori e incompleti**

Per questo molti requisiti vengono in realtà **definiti dagli ingegneri del software**, che devono ottenere informazioni dai soggetti interessati: questo processo si chiama **elicitazione**.

STAKEHOLDER I **soggetti che a vario titolo sono interessati** dalla realizzazione del sistema software. Esempi: utenti finali, manager dell'azienda, personale della manutenzione, esperti di settore, sindacati, ecc.

Durante l'elicitazione, l'ingegnere del software lavora con i clienti per conoscere: il **dominio applicativo**, le **funzionalità** che il sistema dovrà fornire, e i **vincoli operativi** del sistema.

2.2 Requisiti di alto livello vs di dettaglio

L'elicitazione si muove tipicamente su due livelli, dall'alto verso il basso:

Requisiti di alto livello

Si elicitano le **intenzioni e gli obiettivi** dell'azienda e della direzione per il progetto: opportunità di business ed esigenze aziendali, giustificazione del progetto, ambito di applicazione, vincoli principali, funzionalità principali, fattori di successo, caratteristiche degli utenti.

TERMINOLOGIA DA RICORDARE Gli ingegneri del software che devono interagire con la direzione aziendale e gestire questo livello di requisiti sono talvolta chiamati **analisti di business**.

Requisiti di dettaglio

Si elicitano le **esigenze specifiche degli utenti**, organizzate attorno a sei aree che si irradiano da un nucleo comune ("Requisiti"):



Le sei dimensioni verranno riprese in dettaglio nella sezione "Analisi dei requisiti"

2.3 Problematiche di elicitazione

- Gli stakeholder spesso **non sanno di cosa hanno bisogno** davvero
- Gli stakeholder esprimono i requisiti nella **loro terminologia** (non in quella tecnica)
- Diversi stakeholder possono avere **esigenze contrastanti**
- I **fattori organizzativi e politici** dell'azienda possono influenzare i requisiti del sistema
- I requisiti **cambiano durante il processo di analisi** stesso
- Possono emergere **nuovi stakeholder** e l'ambiente di business può cambiare

2.4 Metodi di elicitazione

a) Interviste

Utili per ottenere una comprensione generale di ciò che fanno gli stakeholder e di come potrebbero interagire con il sistema. Due tipi:

- **Interviste chiuse** — basate su un elenco predeterminato di domande
- **Interviste aperte** — in cui vengono esplorate varie questioni insieme agli stakeholder

Indicazioni pratiche per condurle bene:

- Avere una **mentalità aperta**, evitare idee preconcepite sui requisiti ed essere disposti ad ascoltare
- È necessario **stimolare** l'utente a parlare del sistema suggerendo possibili requisiti, piuttosto che chiedere semplicemente "cosa vuoi"

Possibili problemi:

- Gli utenti potrebbero usare un **linguaggio di dominio** difficile da comprendere per l'ingegnere dei requisiti
- Alcune conoscenze sono così **familiari** per gli utenti che questi trovano difficile articularle, o pensano che non valga la pena farlo (*conoscenza tacita*)

b) Analisi etnografica

Un esperto **osserva** accuratamente e analizza il modo in cui le persone lavorano realmente, senza che queste debbano spiegare il proprio lavoro. Permette di cogliere fattori sociali e organizzativi importanti che un'intervista non farebbe emergere.

Punti di forza

- Efficace per comprendere i **processi esistenti** così come sono davvero praticati (non come si dice che siano)

Limite

- **Non è in grado** di identificare nuove funzionalità da aggiungere al sistema — osserva solo ciò che già accade

3 · Analisi dei requisiti

3.1 Da dati grezzi a requisiti organizzati

Dopo che i requisiti sono stati elicitati e raccolti, sono ancora un **insieme non organizzato di dati**. L'analisi dei requisiti riguarda tre compiti:

- **Categorizzazione** dei requisiti
- **Definizione delle priorità** dei requisiti
- La ricerca di **coerenza e completezza** dei requisiti

3.2 Categorizzazione: funzionali vs non funzionali

È la stessa distinzione già introdotta nella Lezione 1, qui approfondita:

Requisiti funzionali	Requisiti non funzionali
Dichiarazioni sulle funzionalità che il sistema dovrebbe fornire, su come dovrebbe reagire a particolari input e su come dovrebbe comportarsi in particolari situazioni. Può indicare anche ciò che il sistema non deve fare.	Vincoli sulle funzionalità offerte (vincoli di tempo, sul processo di sviluppo, sugli standard da adottare, ecc.). Spesso si applicano al sistema nel suo complesso piuttosto che a singole caratteristiche.

ESEMPIO — SISTEMA DI GESTIONE SANITARIA (FUNZIONALI)

- L'utente deve poter effettuare ricerche negli elenchi di appuntamenti per tutte le cliniche
- Il sistema genera ogni giorno, per ogni ambulatorio, l'elenco dei pazienti attesi quel giorno
- Ogni membro del personale deve essere identificato univocamente da un numero di 8 cifre

ESEMPIO — STESSO SISTEMA (NON FUNZIONALI)

- Il sistema deve essere disponibile per tutte le cliniche nel normale orario di lavoro (lun-ven, 08:30-17:30)
- I tempi di inattività in orario di lavoro non devono superare i 5 secondi al giorno
- Il sistema deve implementare le disposizioni sulla privacy dei pazienti previste dalla norma HStan-03-2006-priv

3.3 Evitare le imprecisioni

I requisiti **ambigui** possono essere interpretati in modi diversi da sviluppatori e utenti, con conseguenze concrete:

ESEMPIO DI AMBIGUITÀ REALE Requisito scritto: "L'utente deve essere in grado di effettuare ricerche negli elenchi degli appuntamenti per ogni clinica".
Intenzione dell'utente: inserire il nome del paziente e far cercare il sistema in *tutte* le cliniche contemporaneamente.
Interpretazione degli sviluppatori: l'utente sceglie prima *una* clinica, poi cerca il paziente solo in quell'elenco.
Stesso testo, due sistemi completamente diversi da costruire.

3.4 Le sei dimensioni dei requisiti (categorizzazione di dettaglio)

Una categorizzazione più fine, utile soprattutto in fase di analisi, distingue sei aree — le stesse viste nello schema a raggiera dell'elicitazione, qui con esempi concreti (tratti da un ipotetico sistema bancario/universitario):

1. Funzionalità individuali

Descrivono i singoli servizi offerti dal sistema.

ESEMPI Accesso: login con username e password uniche, password codificate e conservate in modo sicuro. **Gestione del conto:** visualizzare saldo, cronologia transazioni, scaricare estratti conto. **Stampa certificati:** certificato PDF con piano di studi, anno di iscrizione, esami superati (con data e voto).

2. Business flow

Descrivono le interazioni tra uno o più utenti e il sistema per realizzare uno specifico processo aziendale — **aggregano più funzionalità individuali**.

ESEMPI Immatricolazione: lo studente si registra, richiede l'immatricolazione a un corso, la segreteria verifica i requisiti di accesso, il sistema verifica il pagamento delle tasse e completa l'immatricolazione.
Pagamento di una fattura: il richiedente accede, seleziona "Paga fattura", il sistema verifica i fondi disponibili e invia una richiesta di autorizzazione con OTP al responsabile del fondo, che autorizza; il sistema effettua il pagamento, aggiorna la cronologia e notifica via email entrambe le parti.

3. Esigenze di dati e informazioni

Descrivono i dati che il sistema deve gestire.

ESEMPI Dati del profilo utente (nome, indirizzo, contatti, data di nascita); dati sulle transazioni (data, ora, importo, conti di origine/destinazione, tipo); registri di audit (accessi, modifiche, transazioni, conservati per almeno 7 anni).

4. Interfacce utente

Si concentrano su progettazione e usabilità delle interfacce.

ESEMPI Dashboard: riepilogo saldi, transazioni recenti, collegamenti alle funzioni più usate.
Messaggi di errore: devono indicare chiaramente quale informazione era errata e i vincoli da rispettare (es. "Data di pagamento non valida. Devi inserire una data successiva alla data corrente.").

5. Interfacce con sistemi esterni

Definiscono come il sistema interagisce con altri sistemi esterni.

ESEMPI Pagamento tasse tramite gateway **PagoPA**; login tramite **SPID** e Carta di Identità Elettronica (**CIE**); **API** sicura per applicazioni di terze parti che accedono agli orari delle lezioni.

6. Requisiti non funzionali

Qualificano i requisiti funzionali e impongono vincoli al funzionamento del sistema.

ESEMPI Prestazioni: fino a 10.000 utenti simultanei, tempo di risposta medio < 2s.
Sicurezza: dati sensibili cifrati in transito e a riposo, autenticazione a più fattori (MFA).
Affidabilità: uptime 99,9% annuo, failover automatico su server di backup. **Scalabilità:** scalare orizzontalmente aggiungendo server senza tempi di inattività significativi.

Schema di categorizzazione con prefissi

Uno schema pratico per numerare i requisiti nella documentazione:

Area dei requisiti	Prefisso	Esempio di numerazione
Business flow	BF	BF1, BF2, ...
Funzionalità individuale	FI	FI1.1, FI1.2, FI1.3, FI2.1, ...
Esigenza di dati e informazioni	ED	ED1.1, ED1.2, ED2.1, ...
Interfaccia utente	IU	IU1.1, IU1.2, IU2.1, ...
Interfaccia con sistemi esterni	IS	IS1, IS2, ...
Requisiti non funzionali	NF	NF1, NF2, ...

3.5 Priorità dei requisiti

Spesso ci sono limitazioni (tempo, risorse, capacità tecniche) che rendono impossibile sviluppare tutti i requisiti: bisogna dare priorità per capire quali caratteristiche sviluppare per prime. Due criteri principali:

Business value (valore per l'azienda)

Livello	Significato
Alto — "MUST HAVE"	Se il requisito non è soddisfatto, il sistema non è utilizzabile
Medio — "SHOULD HAVE"	Importante, ma non comporta un fallimento se omesso

Rischio tecnico (di realizzazione)

Livello	Significato
Alto	Difficile prevedere se si riuscirà a realizzare il requisito (in assoluto, o nei tempi/costi previsti)
Medio	Si ritiene di poterlo realizzare, ma ci sono fattori di rischio non controllabili
Basso	Piena fiducia nella propria capacità di realizzarlo

Altri criteri per definire le priorità: richieste/esigenze attuali di utenti e clienti, concorrenza e condizioni di mercato, esigenze future dei clienti, problemi critici nel prodotto attuale.

DA RICORDARE Questi criteri sono spesso **soggettivi**: le priorità vanno determinate coinvolgendo **molti stakeholder**, proprio perché punti di vista diversi tendono a dare priorità diverse agli stessi requisiti.

Elenco delle priorità

Uno strumento pratico che aiuta a definire le priorità e a registrare le fonti dei requisiti:

N.	Descrizione sintetica	Fonte	Priorità*	Stato
1	La ricerca su una pagina deve rispondere in meno di 1 secondo	Un rappresentante marketing di un grande cliente	1	Accettato per questa release
2	Il testo di aiuto deve essere sensibile al campo selezionato	Utenti di grandi account	2	Rimandato alla prossima release

*La priorità può essere 1, 2, 3 o 4, dove 1 è la più alta.

3.6 Coerenza e completezza

L'analisi dei requisiti deve anche garantire due proprietà:

- **Completezza** — i requisiti devono includere la descrizione di **tutte** le funzionalità necessarie del sistema
- **Coerenza** — non devono esserci **contraddizioni o conflitti** nelle descrizioni delle funzionalità

A causa della complessità del sistema e dell'ambiente in cui opera, garantire pienamente entrambe spesso diventa quasi impossibile in senso assoluto: restano comunque obiettivi verso cui tendere con la massima cura possibile (da qui l'importanza delle checklist di revisione viste più avanti).

4 · Definizione dei requisiti

4.1 Dalla raccolta alla stesura formale

La **definizione dei requisiti** comporta la loro stesura formale. Si possono usare diversi approcci, spesso **in combinazione tra loro**:

- Linguaggio naturale
- Linguaggio naturale strutturato
- Casi d'uso
- Diagrammi e altre notazioni

4.2 Linguaggio naturale

I requisiti sono scritti come un elenco di descrizioni testuali.

Vantaggi

- Espressivo, intuitivo e universale
- Comprensibile anche da utenti e clienti non tecnici

Svantaggi

- Mancanza di chiarezza (la precisione è difficile)
- Accorpamento di requisiti diversi involontariamente espressi insieme

ESEMPIO — SISTEMA PER MICROINFUSORI DI INSULINA 3.2 Il sistema deve misurare la glicemia e somministrare l'insulina, se necessario, ogni 10 minuti (le variazioni della glicemia sono relativamente lente, quindi non servono misurazioni più frequenti; misurazioni troppo rare potrebbero portare a livelli di glicemia inutilmente elevati).

3.6 Il sistema deve eseguire una routine di autotest ogni minuto, con condizioni da testare e azioni associate definite in una tabella dedicata (una routine di autotest può scoprire problemi hardware/software e avvisare l'utente che il normale funzionamento potrebbe essere compromesso).

4.3 Linguaggio naturale strutturato

I requisiti sono scritti seguendo una **struttura fissa (template)**: ogni campo fornisce informazioni su un aspetto specifico del requisito. Questo riduce l'ambiguità del linguaggio naturale puro, mantenendone la leggibilità.

Tabelle Input-Process-Output (IPO)

N. Requisito	Input	Elaborazione (Process)	Output
--------------	-------	------------------------	--------

12: Ordine cliente	• Articoli per tipo e quantità • Invio della richiesta	• Accettazione degli articoli e delle rispettive quantità	• Messaggio di accettazione • Richiesta messaggio di conferma
--------------------	---	---	--

Esempio di template completo

Un template più ricco, usato per requisiti critici (es. sistemi medicali), può includere campi come nell'esempio seguente (adattato dal caso del microinfusore di insulina):

Identificativo	Insulin Pump/Control Software/SRS/3.3.2
Funzione	Calcolo della dose di insulina: livello di glicemia sicuro
Descrizione	Calcola la dose di insulina da somministrare quando il livello di glicemia misurato è nella zona sicura (tra 3 e 7 unità)
Input	Lettura corrente della glicemia (r2), le due letture precedenti (r0 e r1)
Sorgente	Lettura corrente dal sensore; le altre letture dalla memoria
Output	CompDose — la dose di insulina da somministrare
Destinazione	Ciclo di controllo principale
Azione	CompDose è zero se il livello è stabile o in calo, o se in aumento ma con tasso di crescita in diminuzione. Se il livello aumenta e il tasso aumenta, CompDose si calcola dividendo per 4 la differenza tra livello corrente e precedente, arrotondando il risultato
Richiede	Due letture precedenti, per calcolare il tasso di variazione della glicemia
Precondizione	Il serbatoio di insulina contiene almeno la dose singola massima consentita
Postcondizione	r0 viene sostituito da r1, che viene a sua volta sostituito da r2
Effetti collaterali	Nessuno

4.4 Casi d'uso

I casi d'uso sono utilizzati per identificare:

- Le **funzionalità principali** del sistema
- Le **precondizioni** e le **post-condizioni** delle funzionalità
- Il **flusso di attività** componenti
- Le **condizioni di errore** e i **flussi alternativi**

Saranno approfonditi in una prossima lezione dedicata.

4.5 Tracciabilità dei requisiti

DEFINIZIONE La capacità di **seguire il percorso** dei requisiti durante il processo di sviluppo e di verificare che tutti siano stati sviluppati e testati.

- Ogni requisito deve essere **univocamente identificabile**

- Ogni requisito deve essere collegato alla **fonte documentale o alla persona** che lo ha definito
- Ogni artefatto di progettazione, implementazione e test deve essere collegato al **requisito che lo ha originato**

Matrice di tracciabilità

Aiuta a tenere traccia dei requisiti durante tutto il processo di sviluppo:

Requisiti	Design	Codice	Test	Requisiti correlati
RF1	Componente C1	File F1, F2	Test case T1.1, T1.2, T2.1, T2.2	RF2, RF3
RF2	Componente C1	File F2	Test case T2.3, T2.4	RF1
RF3	Componente C2	File F3	Test case T3.1, T3.2, T3.3	RF1
RF4	...	...	...	...

4.6 Prototipazione dei requisiti

Riguarda principalmente i requisiti dell'interfaccia utente, in termini di:

- **Aspetto visivo** (dimensioni, forme, posizione, colori, stile grafico)
- **Flusso di interazione** (logica di navigazione)

Bassa fedeltà	Alta fedeltà
Schizzi e descrizione del flusso di interazione, testualmente o con diagrammi	Creazione di un'interfaccia preliminare interattiva

ESEMPIO DI PROTOTIPO A BASSA/MEDIA FEDELTA' A lezione viene mostrato un esempio di **wireframe** (realizzato con lo strumento Balsamiq Wireframes): le stesse schermate di un'applicazione vengono disegnate in versione desktop, tablet e mobile usando solo forme geometriche, linee-segnaposto per il testo e riquadri con una "X" al posto delle immagini. L'obiettivo non è mostrare la grafica definitiva, ma validare rapidamente con il cliente **disposizione dei contenuti e flusso di navigazione**, prima di investire tempo nella grafica finale.

4.7 Revisione dei requisiti: checklist

Nove criteri da verificare per ogni requisito prima di considerarlo definitivo:

Criterio	Domanda da porsi
Completezza	I requisiti comprendono tutte le esigenze deducibili dallo scenario?
Coerenza	I requisiti sono privi di contraddizioni tra loro?
Chiarezza	I requisiti sono formulati in modo comprensibile, conciso e preciso?
Univocità	I requisiti sono privi di ambiguità e si prestano a un'unica interpretazione?

Necessarietà	I requisiti riflettono esigenze del cliente, non idee degli sviluppatori su design/ implementazione?
Indipendenza	Ciascun requisito presenta nuove informazioni, non coperte da altri requisiti?
Atomicità	Ciascun requisito specifica una singola funzione o caratteristica?
Fattibilità	I requisiti sono ragionevolmente realizzabili con un effort contenuto?
Verificabilità	Per ciascun requisito è possibile immaginare un test che ne verifichi oggettivamente il soddisfacimento?

5 · Specifica e accettazione dei requisiti

5.1 Specifica dei requisiti (documento SRS)

Una volta elicitati, analizzati, definiti e revisionati, i requisiti devono essere inseriti in un documento di **Specifica dei Requisiti del Software (SRS)**, dall'inglese *Software Requirements Specification*).

Il livello di dettaglio di questo documento dipende da:

- **Dimensione e complessità** del progetto
- **Conoscenza ed esperienza** degli sviluppatori nell'area tematica

5.2 Accettazione dei requisiti

Il documento SRS deve essere **accettato dal cliente**. Questo passaggio ha diverse funzioni:

- Costituisce un **accordo** tra cliente e fornitore
- Può costituire parte di un **contratto formale**
- Serve per **verificare** i risultati intermedi e finali del progetto di sviluppo
- Serve come **base per le richieste di cambiamento** e integrazione successive

IL FILO CONDUTTORE DI TUTTA LA LEZIONE Dall'elicitazione all'accettazione, ogni fase esiste per ridurre lo stesso rischio visto all'inizio: che i requisiti restino **incompleti, ambigui o mal comunicati** tra chi li esprime e chi li deve realizzare. Categorizzazione, priorità, tracciabilità, prototipazione e checklist di revisione sono tutti strumenti concreti per chiudere quel divario *prima* che diventi un problema costoso in fase di implementazione o, peggio, dopo il rilascio (si veda il costo crescente degli errori visto nelle Lezioni 1 e 2).

Riferimento del corso: I. Sommerville, *Ingegneria del Software*, 10ª ed., Pearson — Capitolo 4 (tranne 4.6).

6 · Riepilogo per l'esame

Mappa concettuale in breve

Ingegneria dei requisiti → definire e comprendere funzionalità e vincoli richiesti al sistema (cosa, non come). Un requisito va da astratto (base per un'offerta, aperto all'interpretazione) a formale/dettagliato (base del contratto).

Attività → elicitazione ↔ analisi ↔ [definizione ↓ prototipazione ↓ revisione] ↔ specifica ↔ accordo/accettazione — ciclo con feedback continui, non sequenza rigida. Documentare i requisiti serve a design/implementazione, test, formazione/manutenzione, gestione del progetto. Requisiti incompleti/errati/fraintesi sono causa primaria di fallimento dei progetti.

Elicitazione → coinvolge tutti gli **stakeholder**, non solo l'utente diretto. Requisiti di **alto livello** (obiettivi aziendali) e di **dettaglio** (sei dimensioni: funzionalità individuali, business flow, dati, interfacce utente, interfacce esterne, non funzionali). Problemi tipici: stakeholder che non sanno cosa vogliono, terminologia diversa, esigenze contrastanti. Metodi: interviste (chiuse/aperte) e analisi etnografica (osserva ma non scopre nuove funzionalità).

Analisi → categorizzare (funzionali/non funzionali, sei dimensioni con prefissi BF/FI/ED/IU/IS/NF), dare priorità (business value MUST/SHOULD/NICE-TO-HAVE, rischio tecnico alto/medio/basso), verificare coerenza e completezza. Le ambiguità nel testo dei requisiti generano interpretazioni diverse tra utenti e sviluppatori.

Definizione → linguaggio naturale (espressivo ma ambiguo), linguaggio naturale strutturato (template, tabelle IPO — riduce l'ambiguità), casi d'uso, diagrammi — spesso combinati. Tracciabilità (ogni requisito identificabile e collegato a fonte/design/codice/test tramite matrice). Prototipazione (bassa fedeltà = schizzi, alta fedeltà = interfaccia interattiva). Checklist di revisione a 9 criteri.

Specifiche e accettazione → i requisiti confluiscono nel documento **SRS**, che deve essere accettato dal cliente: diventa accordo/contratto, base di verifica e base per future richieste di cambiamento.

Domande di autoverifica

1. Cosa descrivono i requisiti, e cosa NON descrivono?

→ Descrivono cosa dovrebbe fare il sistema (funzionalità) e sotto quali vincoli deve essere sviluppato e operare. Non descrivono come il sistema dovrà essere realizzato.

2. Quali sono le due funzioni possibili di un requisito, e come cambia il suo livello di dettaglio in ciascun caso?

→ Base per un'offerta/contratto da negoziare (deve restare aperto all'interpretazione) oppure base del contratto stesso (deve essere definito in dettaglio, perché diventa criterio di verifica).

3. Quali attività dell'ingegneria dei requisiti coinvolgono direttamente il cliente/utente?

→ Elicitazione dei requisiti, prototipazione dei requisiti, revisione dei requisiti, accordo e accettazione dei requisiti.

4. Perché non basta chiedere direttamente al cliente/utente quali sono i requisiti?

→ Perché gli utenti spesso comprendono solo i requisiti legati alle proprie mansioni, le motivazioni/obiettivi aziendali non sono sempre chiari a loro, e i requisiti dichiarati possono essere contraddittori e incompleti: per questo molti requisiti vanno elicitati attivamente dagli ingegneri del software presso tutti gli stakeholder.

5. Che differenza c'è tra requisiti di alto livello e requisiti di dettaglio?

→ I requisiti di alto livello elicitano intenzioni e obiettivi dell'azienda/direzione (opportunità di business, ambito, vincoli, funzionalità principali); i requisiti di dettaglio elicitano le esigenze specifiche degli utenti, organizzate in sei dimensioni (funzionalità individuali, business flow, dati, interfacce utente, interfacce esterne, requisiti non funzionali).

6. Quali sono i limiti principali di interviste e analisi etnografica come metodi di elicitazione?

→ Nelle interviste, gli utenti possono usare un linguaggio di dominio difficile da capire o non riuscire ad articolare conoscenze troppo familiari (tacite). L'analisi etnografica è efficace per capire i processi esistenti, ma non è in grado di identificare nuove funzionalità da aggiungere al sistema.

7. Qual è la differenza tra requisiti funzionali e non funzionali, in termini di cosa "vincolano"?

→ I funzionali dichiarano le funzionalità che il sistema deve offrire e come deve comportarsi (anche cosa non deve fare); i non funzionali sono vincoli sulle funzionalità offerte (tempo, sviluppo, standard) e spesso riguardano il sistema nel suo complesso, non singole funzionalità.

8. Perché un requisito ambiguo è pericoloso, anche se sintatticamente corretto?

→ Perché può essere interpretato in modo diverso da chi lo scrive (utente) e da chi lo implementa (sviluppatore), portando a costruire un sistema diverso da quello realmente necessario — come nell'esempio della ricerca "per ogni clinica" interpretata come ricerca simultanea vs ricerca in una clinica selezionata.

9. Quali sono i due criteri principali per definire la priorità di un requisito?

→ Il business value (MUST HAVE / SHOULD HAVE / NICE TO HAVE) e il rischio tecnico di realizzazione (alto / medio / basso).

10. Perché la definizione delle priorità va fatta coinvolgendo molti stakeholder?

→ Perché i criteri di priorità sono spesso soggettivi, e punti di vista diversi possono assegnare priorità diverse agli stessi requisiti.

11. Cosa distingue il linguaggio naturale strutturato dal linguaggio naturale puro nella definizione dei requisiti?

→ Il linguaggio naturale strutturato segue una struttura fissa (template) in cui ogni campo fornisce un'informazione specifica sul requisito (es. tabelle Input-Process-Output), riducendo l'ambiguità tipica del linguaggio naturale libero pur restando leggibile.

12. Cosa serve a garantire la tracciabilità dei requisiti, e con quale strumento si documenta?

→ Serve a seguire il percorso di ogni requisito durante lo sviluppo e verificare che tutti siano stati sviluppati e testati: ogni requisito deve essere identificabile univocamente, collegato alla sua fonte e agli artefatti di design/codice/test che ha originato. Si documenta con la matrice di tracciabilità.

13. Che differenza c'è tra prototipazione a bassa e ad alta fedeltà?

→ Bassa fedeltà: schizzi e descrizione testuale/a diagrammi del flusso di interazione (es. wireframe). Alta fedeltà: creazione di un'interfaccia preliminare realmente interattiva.

14. Elenca almeno cinque dei nove criteri della checklist di revisione dei requisiti.

→ Ad esempio: completezza, coerenza, chiarezza, univocità, necessarietà (più: indipendenza, atomicità, fattibilità, verificabilità).

15. Cos'è il documento SRS e da cosa dipende il suo livello di dettaglio?

→ È il documento di Specifica dei Requisiti del Software in cui confluiscono i requisiti elicitati, analizzati, definiti e revisionati. Il suo livello di dettaglio dipende dalla dimensione/complessità del progetto e dalla conoscenza/esperienza degli sviluppatori nell'area tematica.

16. A cosa serve l'accettazione del documento SRS da parte del cliente?

→ Costituisce un accordo tra cliente e fornitore (eventualmente parte di un contratto formale), serve a verificare i risultati intermedi/finali del progetto e come base per future richieste di cambiamento e integrazione.